

Ad Soyad: _____	Numara: _____	Tarih: _____
-----------------	---------------	--------------

[Genel Hedef] Bu laboratuvarıda Bölüm 11’de inşa ettiğin Zıplayan Macera v3 oyununa *state machine*, *kayıt/yükleme* ve *ayar* sistemlerini ekleyeceksin. Dört çalışma bittiğinde elinde tam donanımlı bir “ZM v4” iskeleti olacak.

1 Çalışma 1: Mini State Machine (Menu - Play - Pause)

Örnek Kod

`calisma1.py` dosyasında üç state’li bir state machine yaz: `MenuState` (ENTER ile başla), `PlayState` (basit hareket ve ESC ile pause) ve `PauseState` (snapshot ile donmuş arka plan).

```

1 import pygame
2
3 GENISLIK, YUKSEKLIK = 800, 600
4
5 class State:
6     def __init__(self, manager): self.manager = manager
7     def gir(self): pass
8     def cik(self): pass
9     def olaylari_isle(self, olaylar): pass
10    def guncelle(self, dt): pass
11    def ciz(self, ekran): pass
12
13 class StateManager:
14    def __init__(self): self._yigin = []
15    @property
16    def aktif(self): return self._yigin[-1] if self._yigin else None
17    def it(self, s): self._yigin.append(s); s.gir()
18    def pop(self):
19        if self._yigin: self._yigin.pop().cik()
20    def degistir(self, s):
21        if self._yigin: self.pop()
22        self.it(s)
23
24 # MenuState, PlayState, PauseState burada...
```

Pause Snapshot

`PauseState` kurucusunda `snapshot=play_state` parametresi al. `gir` metodunda `altyuz = pygame.Surface((W, H))` oluştur, `play_state.ciz(altyuz)` çağır, üstüne karartma ekle. Her karede sadece `ekran.blit(altyuz, (0, 0))` ve “DURAKLATILDI” yazısı.

Talep 1.1: GameOver Ekle

Dördüncü bir state (`GameOverState`) ekle. `PlayState`’te `X` tuşuna basınca `degistir(GameOverState(...))` çağır. `GameOver`’da ENTER yeniden oyunu başlatsın, ESC menüye dönsün.

Talep 1.2: Hayat Sayacı

`PlayState`’e bir `can = 3` ekle. SPACE’e bastıkça `can` azalsın. Sıfıra ulaştığında otomatik `GameOver` state’e geç. `GameOver` ekranında en yüksek skoru veya hayatta kalma süresini göster.

2 Çalışma 2: JSON ile Save/Load

Örnek Kod

`calisma2.py` dosyasında basit bir oyun + JSON save/load. F5 kaydet, F9 yükle. Kayıt dosyası geçici dizinde tutulur.

```
1 import pygame, json, tempfile
2 from pathlib import Path
3
4 class Oyuncu:
5     def __init__(self, x, y, can=5, altin=0):
6         self.x = x; self.y = y
7         self.can = can; self.altin = altin
8
9     def to_dict(self):
10         return {"x": self.x, "y": self.y,
11                "can": self.can, "altin": self.altin}
12
13     @classmethod
14     def from_dict(cls, d):
15         return cls(x=d["x"], y=d["y"],
16                   can=d.get("can", 5),
17                   altin=d.get("altin", 0))
18
19 # Save:
20 def kaydet(yol, oyuncu):
21     with open(yol, "w", encoding="utf-8") as f:
22         json.dump({"version": 1, "oyuncu": oyuncu.to_dict()},
23                 f, indent=2, ensure_ascii=False)
24
25 # Load:
26 def yukle(yol):
27     if not yol.exists(): return None
28     with open(yol, "r", encoding="utf-8") as f:
29         d = json.load(f)
30     return Oyuncu.from_dict(d["oyuncu"])
```

Talep 2.1: Atomik Yazma

`kaydet` fonksiyonunu atomik yazma desenine çevir: önce `.tmp` dosyasına yaz, `f.flush()` + `os.fsync(f.fileno())`, sonra `tmp.replace(yol)`. Test etmek için kaydetme sırasında programı `Ctrl+C` ile kes ve dosyaya bak.

Talep 2.2: Yedek Dosya

Her kaydetmeden önce mevcut dosya varsa `slot.json.bak` olarak yedekle. `yukle` fonksiyonunu güncelle: ana dosya bozuksa (`json.JSONDecodeError`) yedeğe düş. Bozukluğu test etmek için ana dosyaya elinle yanlış metin yaz.

3 Çalışma 3: Çoklu Slot Sistemi + LoadState

Örnek Kod

`calisma3.py` dosyasında tam bir SaveManager + LoadState ekranı. 3 slot, slot başına özet bilgi (ad, seviye, süre, altın), seçim, yükleme, silme.

```
1 class SaveManager:
2     def __init__(self, dizin):
```

```

3     self.dizin = Path(dizin)
4     self.dizin.mkdir(parents=True, exist_ok=True)
5
6     def slot_yolu(self, slot):
7         return self.dizin / f"slot{slot}.json"
8
9     def kaydet(self, slot, veri):
10        # ... atomik + yedek ...
11        pass
12
13    def yukle(self, slot):
14        # ... bozuksa .bak'a dus ...
15        pass
16
17    def slot_ozetleri(self, n=3):
18        sonuc = []
19        for i in range(1, n + 1):
20            yol = self.slot_yolu(i)
21            if not yol.exists():
22                sonuc.append({"slot": i, "bos": True})
23                continue
24            # ... veri oku, ozet cikart ...
25        return sonuc
26
27    def sil(self, slot):
28        yol = self.slot_yolu(slot)
29        if yol.exists(): yol.unlink()
30        bak = yol.with_suffix(".json.bak")
31        if bak.exists(): bak.unlink()
32
33    # LoadState: yukari/asagi sec, ENTER yukle, DEL sil

```

Slot Çerçevesi

Her slot için 90 piksel yüksekliğinde bir çerçeve çiz. Boşsa “-BOS-” yaz, doluysa üç satır: ad, seviye + süre + altın, kayıt zamanı. Seçili slotun çerçevesi beyaz, diğerleri gri olsun.

Talep 3.1: SaveState Ekle

SaveState sınıfı ekle: aynı slot listesi gösterir, ENTER ile seçili slota yazar. Dolu slot üzerine yazılırken “Slot N üzerine yaz?” onayı sor (Y/N tuşları yeterli).

Talep 3.2: Autosave

SaveManager’a autosave_kontrol metodu ekle. PlayState her karede çağırır (oyun zamanını parametre verir). Aralık 5 sn olsun. Autosave gerçekleştiğinde sağ üstte 2 sn süreyle “Otomatik kayıt yapıldı” yazısı belirsin (sönen efekt).

4 Çalışma 4: Settings Sınıfı + Ayar Menüsü

Örnek Kod

calisma4.py dosyasında Settings sınıfı ve oyun içi ayar menüsü. Slider, toggle, seçim ve tuş atama wid- get’ları.

```

1  DEFAULTS = {
2      "version": 1,
3      "ses_genel": 1.0, "ses_muzik": 0.7, "sessiz": False,
4      "tam_ekran": False,
5      "tus_sol": pygame.K_LEFT, "tus_sag": pygame.K_RIGHT,
6      "tus_zipla": pygame.K_SPACE,

```

```

7     "zorluk": "orta",
8 }
9
10 class Settings:
11     def __init__(self, yol):
12         self.yol = Path(yol)
13         self._veri = dict(DEFAULTS)
14         self.yukle()
15
16     def yukle(self):
17         if not self.yol.exists(): return
18         try:
19             with open(self.yol, "r", encoding="utf-8") as f:
20                 kullanici = json.load(f)
21         except json.JSONDecodeError: return
22         for k, v in kullanici.items():
23             if k in DEFAULTS: self._veri[k] = v
24
25     def kaydet(self):
26         with open(self.yol, "w", encoding="utf-8") as f:
27             json.dump(self._veri, f, indent=2,
28                       ensure_ascii=False, sort_keys=True)
29
30     def __getitem__(self, k): return self._veri[k]
31     def __setitem__(self, k, v):
32         self._veri[k] = v; self.kaydet()
33
34 # AYAR_SATIRLARI = [...]
35 # class SettingsState: ...

```

Slider Çizimi

Slider için: `bar = pygame.Rect(380, y+8, 250, 8)` taban olarak gri, `dolu = pygame.Rect(380, y+8, doluluk, 8)` renkli. Doluluk değeri: `int(250 * (deger - min) / (max - min))`.

Talep 4.1: Tuş Çakışma Kontrolü

Tuş atama modunda yeni atanmak istenen tuş başka bir eyleme atanmışsa “Bu tus zaten atanmış: {eylem}” uyarısı 3 saniye süreyle göster ve atamayı iptal et. Aynı eyleme aynı tuşu yeniden atamak izinli olsun.

Talep 4.2: Anında Uygulama

Slider değişimlerinde ses ayarları için `pygame.mixer.music.set_volume(...)` anında çağrılmalı — oyuncu kaydıcıyı sürüklerken sesi anlık duysun. Tam ekran toggle değiştiğinde `pygame.display.set_mode(..., pygame.FULLSCREEN)` ile pencere yeniden boyutlandırılmalı.

Talep 4.3: Sıfırlama

Menüye “Sıfırla” satırı ekle (en altta, kırmızı renkte). ENTER’a basıldığında “Tum ayarlar varsayılanına döner. Emin misin? (Y/N)” onayı sor. Y ile `settings._veri = dict(DEFAULTS)` ve `kaydet()` çağrılmalı. N ile geri dön.

5 Bonus: Birleştirme

Bonus Talep

Dört çalışmayı tek bir [ana.py](#) dosyasında birleştir:

- StateManager ile akış: MenuState → PlayState (load/save) → PauseState → SettingsState
- Menüden “Yukle” seçince LoadState
- PauseState’ten “Ayarlar” seçince SettingsState yığına itilir
- Settings.json ile ayarlar, slot1-3.json ile kayıtlar
- Autosave 5 saniyede bir slot 0’a yazar

Bu, Zıplayan Macera v4’ün çekirdek mimarisidir.